

Phase 0 Fine-Tuning Roadmap

A Weekend-to-Two-Week Project on RunPod.io

Manoj Chandrashekar Rao
AI Engineer

June 13, 2026

Contents

1	Executive Summary	4
2	What You Will Learn	4
3	Recommended RunPod Configuration	4
3.1	GPU Choices	4
3.2	Storage	5
3.3	Template Selection	5
4	Overall Pipeline	5
5	Milestone 1: Environment Setup	6
5.1	Steps	6
5.2	What You Learn	6
5.3	Success Criteria	7
6	Milestone 2: Dataset Preparation	7
6.1	Dataset Choice	7
6.2	Steps	7
6.3	What You Learn	8
6.4	Success Criteria	8
7	Milestone 3: Baseline Evaluation	8
7.1	Benchmarks	8
7.2	Steps	8
7.3	What You Learn	9
7.4	Success Criteria	9
8	Milestone 4: QLoRA Fine-Tuning	9
8.1	Concept	9
8.2	Steps	10
8.3	What You Learn	11
8.4	Success Criteria	12
9	Milestone 5: Merge Adapters	12
9.1	Why Merge?	12
9.2	Steps	12
9.3	What You Learn	12
9.4	Success Criteria	13
10	Milestone 6: Quantize to GGUF	13
10.1	Steps	13
10.2	What You Learn	13
10.3	Success Criteria	13
11	Milestone 7: Final Evaluation and Hosting	14
11.1	Steps	14
11.2	What You Learn	14
11.3	Success Criteria	15
12	Two-Week Schedule	15

13 Common Pitfalls and How to Avoid Them	15
14 Budget Estimate	15
15 From Phase 0 to Phase 1 (MiniMax-M3)	16
16 Summary Checklist	16

1 Executive Summary

This document is a complete, hands-on roadmap for **Phase 0** of your larger goal: learning to fine-tune open-weights models so that you can later tackle MiniMax-M3. Instead of jumping straight into a 428B-parameter model, Phase 0 uses a **7B-parameter coding model** to teach the full pipeline in a controlled, affordable way.

Target model: [Qwen2.5-Coder-7B-Instruct](#)

Why this model?

- Small enough to fine-tune on a single GPU.
- Strong enough to show meaningful before/after differences.
- Apache 2.0 license — no commercial restrictions.
- Excellent tokenizer and code corpus.
- Widely supported by HuggingFace, Unsloth, llama.cpp, and vLLM.

Platform: RunPod.io Secure Cloud.

Estimated time: One focused weekend (16–20 hours) or two relaxed weekends.

Estimated cost: \$10–\$50 on RunPod, depending on GPU choice.

2 What You Will Learn

By the end of Phase 0, you will have first-hand experience with:

1. Provisioning and configuring a GPU cloud instance.
2. Preparing a coding dataset in the correct chat-template format.
3. Running baseline evaluation on HumanEval and MBPP.
4. Fine-tuning with QLoRA adapters.
5. Merging adapters back into the base model.
6. Quantizing a fine-tuned model to GGUF.
7. Re-evaluating the quantized model and uploading it to HuggingFace.

3 Recommended RunPod Configuration

3.1 GPU Choices

For a 7B model with QLoRA, the following RunPod GPUs are suitable. Table 1 summarizes options.

Recommendation: Start with an **RTX A6000** or **A100 40 GB**. The A6000 gives the best cost/comfort ratio; the A100 trains faster and leaves more headroom for experiments.

Table 1: RunPod GPU options for Phase 0

GPU	VRAM	Approx. Speed	Cost/Hour
RTX 3090	24 GB	Medium	\$0.44–\$0.69
RTX 4090	24 GB	Fast	\$0.69–\$0.99
RTX A6000	48 GB	Medium-Fast	\$0.79–\$0.99
A100 40 GB	40 GB	Fast	\$1.99–\$2.49
A100 80 GB	80 GB	Fastest	\$2.99–\$3.99

3.2 Storage

Request at least **100 GB persistent network volume**. You will need space for:

- Base model (~15 GB)
- Adapter checkpoints (~1–2 GB)
- Merged model (~15 GB)
- GGUF quantus (~4–8 GB each)
- Datasets and cache (~10–20 GB)

3.3 Template Selection

In RunPod:

1. Choose **Secure Cloud**.
2. Select **PyTorch** template (e.g., “PyTorch 2.3 + CUDA 12.1”).
3. Mount a **Network Volume** so your data survives pod restarts.
4. Expose **HTTP ports** if you want to test inference via a simple UI later.

4 Overall Pipeline

Figure 1 shows the complete Phase 0 pipeline.

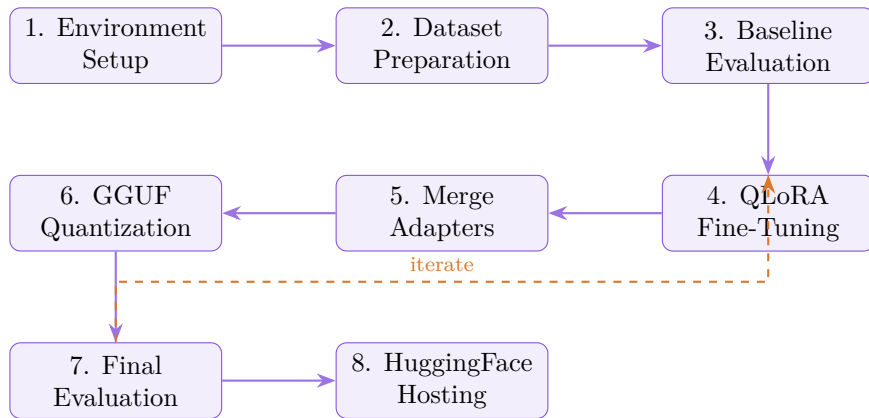


Figure 1: Phase 0 end-to-end fine-tuning pipeline

5 Milestone 1: Environment Setup

Goal: Have a working RunPod pod with all required libraries installed.

Estimated time: 1–2 hours.

5.1 Steps

1. Create a RunPod pod.

- GPU: RTX A6000 or A100 40 GB.
- Template: PyTorch 2.3+ / CUDA 12.1+.
- Network volume: 100 GB minimum.

2. Connect via JupyterLab or SSH. RunPod provides both. SSH is better for long-running training jobs.

3. Create a working directory.

```
mkdir -p /workspace/phase0-finetune
cd /workspace/phase0-finetune
```

4. Install dependencies.

```
pip install --upgrade pip
pip install torch torchvision torchaudio --index-url \
    https://download.pytorch.org/whl/cu121
pip install transformers datasets accelerate peft trl \
    bitsandbytes huggingface-hub evaluate
pip install lm-eval
pip install unsloth
```

5. Log in to HuggingFace. Get a token from huggingface.co/settings/tokens.

```
huggingface-cli login
```

6. Verify GPU.

```
nvidia-smi
python -c "import torch; print(torch.cuda.get_device_name(0))"
```

5.2 What You Learn

- How to provision cloud GPUs.
- Which Python packages form the modern fine-tuning stack.
- How to authenticate with HuggingFace from a remote machine.

5.3 Success Criteria

- `nvidia-smi` shows your GPU.
- `import torch; torch.cuda.is_available()` returns `True`.
- You can run `huggingface-cli whoami` and see your username.

6 Milestone 2: Dataset Preparation

Goal: Build a high-quality coding dataset in the model's chat template format.

Estimated time: 2–4 hours.

6.1 Dataset Choice

For a first project, use a publicly available coding instruction dataset. Two good options:

- [python_code_instructions_18k_alpaca](#)
- [CodeAlpaca-20k](#)
- [code_instructions_122k_alpaca_style](#) (if you want more data)

6.2 Steps

1. **Download the dataset.**

```
from datasets import load_dataset
dataset = load_dataset("iamtarun/python_code_instructions_18k_alpaca",
                      split="train")

print(dataset)
print(dataset[0])
```

2. **Inspect the schema.** Typical fields: `instruction`, `input`, `output`. You need to map these to `messages` or a single prompt string.
3. **Format with the chat template.** Qwen2.5-Coder uses the standard Qwen chat template.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained(
    "Qwen/Qwen2.5-Coder-7B-Instruct",
    trust_remote_code=True
)

def format_example(example):
    instruction = example["instruction"]
    if example.get("input"):
        instruction += "\n" + example["input"]
    messages = [
        {"role": "system", "content": "You are a helpful coding assistant."},
        {"role": "user", "content": instruction},
        {"role": "assistant", "content": example["output"]},
    ]
```

```

    ]
    return {"text": tokenizer.apply_chat_template(
        messages, tokenize=False, add_generation_prompt=False)}

formatted = dataset.map(format_example)
formatted = formatted.train_test_split(test_size=0.05, seed=42)
formatted.save_to_disk("/workspace/phase0-finetune/data")

```

4. Verify token lengths.

```

lengths = [len(tokenizer(x)["input_ids"]) for x in formatted["train"]["text"]]
print("Max:", max(lengths))
print("Mean:", sum(lengths)/len(lengths))

```

Keep max sequence length < 2048 for faster training.

6.3 What You Learn

- Why chat templates matter.
- How instruction-tuning data differs from pre-training data.
- How to split, tokenize, and cache datasets efficiently.

6.4 Success Criteria

- You have a `train` and `test` split on disk.
- A sample `text` field looks like a natural conversation with `<|im_start|>` and `<|im_end|>` tokens.
- Token length distribution is reasonable for your GPU budget.

7 Milestone 3: Baseline Evaluation

Goal: Measure the base model's coding ability before fine-tuning.

Estimated time: 2–3 hours.

7.1 Benchmarks

Use **HumanEval** and **MBPP**. These are the standard coding benchmarks.

7.2 Steps

1. Run HumanEval with lm-eval-harness.

```

lm_eval --model hf \
  --model_args pretrained=Qwen/Qwen2.5-Coder-7B-Instruct,dtype=bfloat16 \
  --tasks humaneval \
  --batch_size 1 \
  --output_path ./baseline_eval

```


2. Run MBPP.

```
lm_eval --model hf \
  --model_args pretrained=Qwen/Qwen2.5-Coder-7B-Instruct,dtype=bfloat16 \
  --tasks mbpp \
  --batch_size 1 \
  --output_path ./baseline_eval
```

3. Record the scores. You want a table like Table 2.

Table 2: Example baseline results sheet

Benchmark	pass@1
HumanEval	??.%
MBPP	??.%

7.3 What You Learn

- How to run standardized LLM benchmarks.
- Why pass@1 matters for coding.
- How to compare models fairly.

7.4 Success Criteria

- You have baseline numbers written down.
- The evaluation completed without out-of-memory errors.

8 Milestone 4: QLoRA Fine-Tuning

Goal: Train LoRA adapters on top of the quantized base model.

Estimated time: 4–8 hours.

8.1 Concept

QLoRA keeps the base model frozen in 4-bit precision and trains small, low-rank adapter matrices. This is the standard way to fine-tune large models on modest hardware.

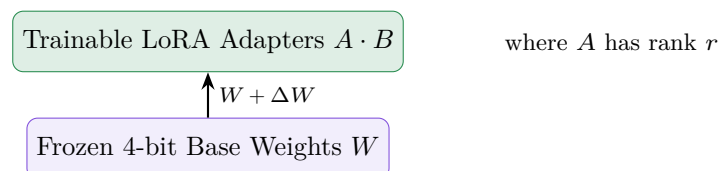


Figure 2: QLoRA: trainable adapters on a frozen quantized base

8.2 Steps

1. Create a training script train.py.

```
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    BitsAndBytesConfig,
    TrainingArguments,
)
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
from trl import SFTTrainer
from datasets import load_from_disk

# Load tokenizer
tokenizer = AutoTokenizer.from_pretrained(
    "Qwen/Qwen2.5-Coder-7B-Instruct",
    trust_remote_code=True
)
tokenizer.pad_token = tokenizer.eos_token

# 4-bit quantization config
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype="bfloat16",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
)

# Load model
model = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-Coder-7B-Instruct",
    quantization_config=bnb_config,
    device_map="auto",
    trust_remote_code=True,
)
model = prepare_model_for_kbit_training(model)

# LoRA config
lora_config = LoraConfig(
    r=64,
    lora_alpha=128,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
)
model = get_peft_model(model, lora_config)

# Load data
dataset = load_from_disk("/workspace/phase0-finetune/data")
```

```

# Training arguments
args = TrainingArguments(
    output_dir="/workspace/phase0-finetune/qwen-coder-lora",
    num_train_epochs=3,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    max_grad_norm=0.3,
    warmup_ratio=0.03,
    lr_scheduler_type="cosine",
    logging_steps=10,
    save_strategy="epoch",
    bf16=True,
    optim="paged_adamw_8bit",
    group_by_length=True,
    report_to="none",
)

# Trainer
trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
    args=args,
    max_seq_length=2048,
    dataset_text_field="text",
)

# Train
trainer.train()

# Save adapters
model.save_pretrained("/workspace/phase0-finetune/qwen-coder-lora-final")

```

2. Run training.

```
python train.py
```

3. **Monitor loss.** It should decrease smoothly. If it spikes, lower the learning rate.

8.3 What You Learn

- How 4-bit quantization and LoRA interact.
- How to configure rank, alpha, target modules, and learning rate.
- How to monitor training loss and spot overfitting.

8.4 Success Criteria

- Training completes all epochs without OOM.
- Training loss decreases and eval loss stays stable or improves slightly.
- Adapter files are saved under `qwen-coder-lora-final/`.

9 Milestone 5: Merge Adapters

Goal: Combine the trained adapters with the base model to create a single fine-tuned model.

Estimated time: 1–2 hours.

9.1 Why Merge?

Merging lets you:

- Run inference without the PEFT wrapper.
- Quantize to GGUF in one step.
- Upload a complete model to HuggingFace.

9.2 Steps

```
from peft import PeftModel
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

base = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-Coder-7B-Instruct",
    torch_dtype=torch.bfloat16,
    device_map="auto",
    trust_remote_code=True,
)
tokenizer = AutoTokenizer.from_pretrained(
    "Qwen/Qwen2.5-Coder-7B-Instruct",
    trust_remote_code=True
)

model = PeftModel.from_pretrained(
    base,
    "/workspace/phase0-finetune/qwen-coder-lora-final",
)
model = model.merge_and_unload()

model.save_pretrained("/workspace/phase0-finetune/qwen-coder-merged")
tokenizer.save_pretrained("/workspace/phase0-finetune/qwen-coder-merged")
```

9.3 What You Learn

- What adapter merging mathematically means: $W_{final} = W_{base} + A \cdot B$.
- When to merge vs. keep adapters separate.

9.4 Success Criteria

- Merged model directory contains `config.json`, `model.safetensors`, and tokenizer files.
- You can load the merged model with `AutoModelForCausalLM.from_pretrained`.

10 Milestone 6: Quantize to GGUF

Goal: Convert the merged fine-tuned model into a quantized GGUF file for efficient inference.

Estimated time: 2–3 hours.

10.1 Steps

1. Clone llama.cpp.

```
git clone https://github.com/ggerganov/llama.cpp.git
cd llama.cpp
pip install -r requirements.txt
```

2. Convert to FP16 GGUF.

```
python convert_hf_to_gguf.py \
    /workspace/phase0-finetune/qwen-coder-merged \
    --outfile /workspace/phase0-finetune/qwen-coder-f16.gguf
```

3. Quantize to Q4_K_M.

```
./llama-quantize \
    /workspace/phase0-finetune/qwen-coder-f16.gguf \
    /workspace/phase0-finetune/qwen-coder-q4_k_m.gguf \
    Q4_K_M
```

4. (Optional) Also quantize to Q5_K_M for higher quality.

10.2 What You Learn

- The difference between FP16, Q4_K_M, and Q5_K_M quantization.
- How GGUF files store both weights and tokenizer metadata.
- Trade-off between file size and accuracy.

10.3 Success Criteria

- GGUF file is created and its size matches expectations:
 - Q4_K_M: ~4.4 GB for a 7B model
 - Q5_K_M: ~5.3 GB
- You can run it with `llama-cli` or `llama-server`.

11 Milestone 7: Final Evaluation and Hosting

Goal: Measure the fine-tuned model and upload it to HuggingFace.

Estimated time: 2–3 hours.

11.1 Steps

1. Evaluate the merged bf16 model.

```
lm_eval --model hf \
  --model_args pretrained=/workspace/phase0-finetune/qwen-coder-merged,
  dtype=bfloat16 \
  --tasks humaneval,mbpp \
  --batch_size 1 \
  --output_path ./finetuned_eval
```

2. Record results in Table 3.

3. Upload to HuggingFace.

```
from huggingface_hub import HfApi
api = HfApi()
api.create_repo(repo_id="your-username/qwen2.5-coder-7b-finetuned",
                private=False)
api.upload_folder(folder_path="/workspace/phase0-finetune/qwen-coder-merged",
                  repo_id="your-username/qwen2.5-coder-7b-finetuned")
```

4. Upload GGUF separately (optional but useful).

```
api.create_repo(repo_id="your-username/qwen2.5-coder-7b-finetuned-gguf")
api.upload_file(path_or_fileobj="qwen-coder-q4_k_m.gguf",
                path_in_repo="qwen-coder-q4_k_m.gguf",
                repo_id="your-username/qwen2.5-coder-7b-finetuned-gguf")
```

Table 3: Results comparison table to fill in

Model	HumanEval	MBPP	Size
Base Qwen2.5-Coder-7B	??.%	??.%	15 GB
Fine-tuned (bf16)	??.%	??.%	15 GB
Fine-tuned (Q4_K_M GGUF)	??.%	??.%	4.4 GB

11.2 What You Learn

- How to compare base vs. fine-tuned vs. quantized models.
- How to publish models on HuggingFace.
- How to write a useful model card.

11.3 Success Criteria

- You have a before/after comparison table.
- Your model is live on HuggingFace.
- The model card explains the base model, dataset, training config, and license.

12 Two-Week Schedule

Figure 3 visualizes one way to spread the work over two weekends.

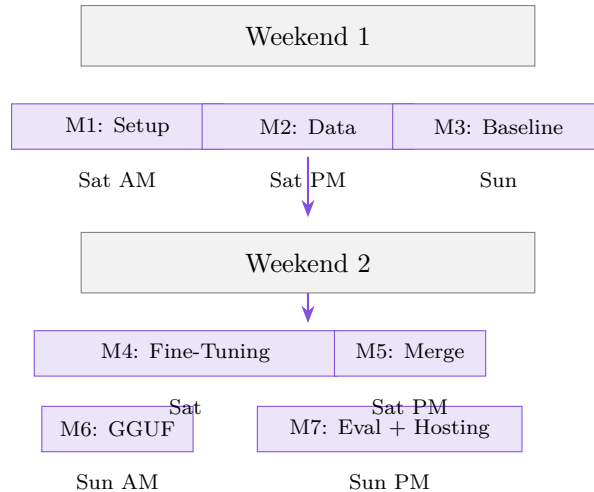


Figure 3: Suggested two-weekend schedule

13 Common Pitfalls and How to Avoid Them

1. **Out-of-memory during training.** Fix: reduce batch size, increase gradient accumulation, lower `max_seq_length`, or use a smaller rank.
2. **Loss not decreasing.** Fix: lower learning rate, check data formatting, ensure labels mask padding and the prompt portion.
3. **Overfitting.** Fix: use a validation split, reduce epochs, add dropout, or use a smaller rank.
4. **Chat template mismatch.** Fix: always use `tokenizer.apply_chat_template` and inspect the resulting string.
5. **GGUF conversion fails.** Fix: ensure the merged model is in FP16/BF16, not still a PEFT model.
6. **RunPod pod shuts down and work is lost.** Fix: always save data and checkpoints to the **network volume**, not the ephemeral pod disk.

14 Budget Estimate

Table 4 gives a rough cost estimate for the two-week project.

Training (Milestone 4) consumes most of the hours. You can reduce cost by spot-checking on a cheap GPU and running the final training on a fast GPU for fewer hours.

Table 4: RunPod cost estimate for Phase 0		
GPU	Hours	Total Cost
RTX 3090 (24 GB)	16–20	\$7–\$14
RTX A6000 (48 GB)	16–20	\$13–\$20
A100 40 GB	12–16	\$24–\$40

15 From Phase 0 to Phase 1 (MiniMax-M3)

Once Phase 0 is complete, the same pipeline applies to MiniMax-M3, but at a much larger scale:

- You will need multi-GPU nodes or very high RAM.
- Dataset quality becomes even more important.
- Evaluation will be slower and more expensive.
- You will use the same tools (QLoRA, transformers, llama.cpp) but with larger config values.

Figure 4 compares the hardware needed for Phase 0 vs. Phase 1.

Phase 0: Qwen2.5-Coder-7B

 Single GPU (24–48 GB VRAM)

Phase 1: MiniMax-M3 (428B)

 Multi-GPU node or 200GB+ unified memory

Figure 4: Relative hardware scale: Phase 0 vs. Phase 1

16 Summary Checklist

Before you start Phase 1, make sure you can confidently check every box:

- ☐ Provisioned a RunPod GPU pod.
- ☐ Prepared and tokenized a coding dataset.
- ☐ Ran baseline evaluation on HumanEval/MBPP.
- ☐ Fine-tuned a model with QLoRA.
- ☐ Merged adapters into a full model.
- ☐ Converted the model to GGUF.
- ☐ Re-evaluated and uploaded the model to HuggingFace.
- ☐ Documented results and learned from failures.

Good luck with Phase 0. Once this pipeline feels routine, MiniMax-M3 becomes a scaling problem rather than a mystery.